

FLAG ARAMA VE REGEX KARAKTER KÜMELERİ

[Kaynak](#)

İÇERİK

1. Düzenli İfadeler (Regex) ve Karakter Kümesi (Charsets) Kullanımı
2. Karakter Grupları ve Tekrarlar
3. Konum Belirleyiciler ve Gruplandırma

Düzenli İfadeler (Regex) ve Karakter Kümesi (Charsets) Kullanımı

1. Giriş: String Arama ve Desen (Pattern) Eşleştirme

Dosya veya metin blokları içinde arama yaparken, `grep 'string' <dosya>` komutu gibi sabit dizeleri aramak temel bir işlemdir. Ancak gerçek dünya senaryolarında belirli bir kelimeyi değil, bir **metin desenini (pattern)** arama ihtiyacı doğar.

- **Problem:** Belirli bir harfle başlayan tüm kelimeleri bulmak veya sonunda rakam olan kelimeleri ayıklamak.
- **Çözüm:** Bu tür karmaşık arama gereksinimleri için **Düzenli İfadeler (Regular Expressions)** ve bunların temel taşı olan **Karakter Kümeleri (Charsets)** kullanılır.

2. Karakter Kümeleri (Charsets) Nedir?

Karakter kümeleri, köşeli parantez `[ ]` kullanılarak tanımlanır. Bu parantezlerin içine yazılan karakterler, aranacak metinde eşleşmesi istenen karakter veya karakter aralıklarını temsil eder.

- **Temel İşleyiş:** Tanımlanan küme metin içindeki **her bir eşleşmeyi** tek tek bulur.
- **Örnek 1:** `[abc]` ifadesi; 'a', 'b' veya 'c' harflerinden herhangi birini gördüğü her yerde eşleşme sağlar.
- **Örnek 2:** `[abc]zz` ifadesi; 'azz', 'bzz' ve 'czz' dizelerini yakalar.

3. Karakter Aralıklarını Tanımlama (Ranges)

Uzun karakter listelerini tek tek yazmak yerine, tire `-` sembolü kullanılarak aralıklar tanımlanabilir.

- **Sıralı Harf Aralıkları:** `[a-c]zz` ifadesi, `[abc]zz` ile aynı sonucu verir.
- **Kombine Aralıklar:** Farklı aralıklar aynı parantez içinde birleştirilebilir.

- `[a-cx-z]zz` ifadesi; hem 'a, b, c' ile başlayan hem de 'x, y, z' ile başlayan ve sonu 'zz' ile biten dizeleri (azz, bzz, czz, xzz, yzz, zzz) yakalar.
- **Alfabenin Tamamı:** `[a-zA-Z]` ifadesi, küçük veya büyük fark etmeksizin **tek bir harf** ile eşleşir.
- **Rakam Kullanımı:** `file[1-3]` ifadesi; 'file1', 'file2' ve 'file3' dosyalarını hedeflemek için kullanılır.

4. Karakterleri Hariç Tutma (Exclusion)

Bazı durumlarda, belirli karakterler dışındaki her şeyle eşleşmek isteyebiliriz. Bu durumda şapka `^` sembolü küme içinde kullanılır.

- **Tek Karakteri Hariç Tutma:** `[^k]ing` ifadesi; 'ring', 'sing' veya '\$ing' kelimelerini bulur ancak 'king' kelimesini **reddeder**.
- **Küme Halinde Hariç Tutma:** `[^a-c]at` ifadesi; 'fat' veya 'hat' kelimeleriyle eşleşirken, baş harfi a, b veya c olan 'bat' veya 'cat' kelimelerini kapsamaz.

5. Kritik Notlar ve Uygulama Stratejileri

Önemli Ayrım (Dizeler vs. Kümeler): Karakter kümelerini (`[abc]`) düz bir metin dizisiyle ("abc") karıştırmayın. `[abc]` kümesi, metin içindeki 'a', 'b' ve 'c' harflerinin **her bir oluşumuyla** ayrı ayrı eşleşir. Örneğin "cba" veya "ca" metinleri de bu küme tanımına uygundur; çünkü küme, sıralamaya değil karakterin varlığına bakar.

Regex Yazarken Verimlilik Kuralları

Aynı sonucu veren birden fazla Regex yazılabilir. Ancak akademik ve teknik açıdan en doğru yaklaşım **verimli (efficient)** olanı seçmektir:

1. **Spesifik Olun (Gereksiz Kapsamdan Kaçının):** Eğer soruda sadece 'a' ve 'c' arasındaki karakterler isteniyorsa, `[a-z]` (tüm alfabe) yerine `[a-c]` kullanmalısınız.
2. **Aşırı Detaydan Kaçının (Basitlik İlkesi):** Eğer eşleşmesi gereken karakter sayısı çok fazlaysa (örneğin: a, c, f, r, s, z gibi dağınık harfler), bunları tek tek yazıp karmaşıklık yaratmak yerine `[a-z]` gibi daha kısa ve genel bir ifade kullanmak daha mantıklıdır.

Düzenli İfadelerde Joker ve İsteğe Bağlı Karakterler

Düzenli ifadelerle (Regex) çalışırken, tam olarak ne olduğunu bilmediğimiz karakterleri yakalamak veya bazı karakterlerin varlığını esnek hale getirmek için belirli semboller kullanılır. Bu notta, bu sembollerin en kritikleri olan "Nokta" ve "Soru İşareti" operatörlerini inceleyeceğiz.

1. Herhangi Bir Tek Karakterin Yakalanması: Nokta (`.`) Operatörü

Regex dilinde nokta karakteri bir **wildcard** (joker) görevi görür. Satır sonu karakteri (line break) hariç, karşısına çıkan herhangi bir tek karakterle eşleşir.

- **Temel Mantık:** Belirli bir pozisyonda ne tür bir karakter (harf, rakam, sembol, boşluk) olduğunun bir önemi yoksa nokta kullanılır.
- **Kullanım Senaryosu:** `a.c` deseni üzerinden inceleyelim:
 - Bu desen, başında "a", sonunda "c" olan ve arasında **herhangi bir karakter** bulunan 3 karakterli tüm dizileri yakalar.
 - **Eşleşen Örnekler:** `aac` , `abc` , `a0c` , `a!c` , `a c` .
- **Sınırlandırma:** Nokta operatörü tek başına kullanıldığında sadece **bir adet** karakteri temsil eder. Eğer iki karakter yakalanmak istenirse `..` şeklinde kullanılmalıdır.

2. İsteğe Bağlı Karakter Belirleme: Soru İşareti (?) Operatörü

Bir karakterin metin içerisinde "olsa da olur, olmasa da olur" durumunu belirtmek için soru işareti kullanılır. Bu, deseninize esneklik kazandırır.

- **Temel Mantık:** Soru işareti, kendisinden **hemen önce gelen** karakteri opsiyonel hale getirir. Yani o karakter metinde ya 0 kez ya da 1 kez bulunmalıdır.
- **Kullanım Senaryosu:** `abc?` deseni üzerinden inceleyelim:
 - Buradaki `?` , sadece `c` karakterini etkiler.
 - **Eşleşen Örnekler:** `* ab` (c karakteri 0 kez var, yani yok).
 - `abc` (c karakteri 1 kez var).
- **Neden Önemlidir?** Özellikle kelimelerin farklı yazımları (örneğin İngiliz İngilizcesi vs. Amerikan İngilizcesi: `color` ve `colour`) veya çoğul eklerinin (s takısı gibi) kontrolünde hayati rol oynar.

3. Kaçış Karakteri (Escaping) ve Literal (Gerçek) Nokta Kullanımı

Bazen metin içerisinde gerçekten bir "nokta" karakterini aramak isteyebilirsiniz (örneğin bir dosya uzantısını veya cümle sonunu bulmak için). Ancak doğrudan `.` yazarsanız, Regex bunu "herhangi bir karakter" olarak algılayacaktır.

- **Ters Bölü (\) Kullanımı:** Bir meta karakterin (özel anlamı olan sembolün) özel anlamını iptal edip onu sıradan bir metin karakterine dönüştürmek için başına `\` eklenir.
- **Kritik Fark ve Karşılaştırma:**
 - **Desen:** `a.c`
 - **Sonuç:** `a.c` dizisini yakalar ancak aynı zamanda `abc` , `a@c` , `a1c` gibi dizileri de yakalar. Bu durum hatalı eşleşmelere (false positive) yol açabilir.
 - **Desen:** `a\.c`
 - **Sonuç:** **Sadece** içinde gerçek bir nokta olan `a.c` dizisini yakalar. Diğer hiçbir karakterle eşleşmez.

Özet Tablo

Karakter	İşlevi	Örnek Desen	Eşleşme Örneği
.	Herhangi bir tek karakteri temsil eder.	p.n	pan , p&n , p2n
?	Kendinden önceki karakteri opsiyonel yapar.	apple?	appl , apple
\.	Sadece gerçek (literal) nokta karakterini arar.	v1\.0	Sadece v1.0

Düzenli İfadelerde (Regex) Karakter Grupları ve Tekrarlar

Düzenli ifadelerle çalışırken büyük karakter kümelerini (charset) manuel olarak yazmak yerine, belirli karakter türlerini temsil eden **metakarakterler** kullanılır. Bu yapılar, karmaşık dizeleri çok daha hızlı ve hatasız bir şekilde eşleştirmeyi sağlar.

1. Karakter Grubu Kısayolları (Metacharacters)

Metakarakterler, belirli bir karakter tipini (sayı, harf, boşluk vb.) temsil eden özel sembollerdir. Harfin küçük veya büyük olması, ifadenin anlamını tam tersine çevirir.

- **\d (Digit):** Tek bir rakamla eşleşir. (Örnek: 9 , 0 , 5)
- **\D (Non-digit):** Rakam olmayan tek bir karakterle eşleşir. (Örnek: A , @ , ! , ,)
- **\w (Alphanumeric):** Alfanoümerik bir karakterle (harfler ve sayılar) eşleşir.
 - **Önemli Not:** Alt çizgiler (_) de \w grubuna dahildir. Dolayısıyla test_file içerisindeki tüm karakterler \w ile eşleşir.
- **\W (Non-alphanumeric):** Alfanoümerik olmayan karakterlerle eşleşir. (Örnek: ! , # , boşluk karakterleri)
- **\s (Whitespace):** Boşluk karakterleri ile eşleşir. Buna standart boşluklar, sekmeler (tabs) ve satır sonları (line breaks) dahildir.
- **\S (Non-whitespace):** Boşluk dışındaki her şeyle (harfler, rakamlar, semboller) eşleşir.

2. Tekrarlar ve Nicelik Belirleyiciler (Repetitions)

Çoğu zaman belirli bir karakter tipinden arka arkaya kaç tane geldiğini belirtmemiz gerekir. Tekrarlar, kendisinden önce gelen karakterin, metakarakterin veya karakter kümesinin kaç kez tekrarlanacağını tanımlar.

Belirli Sayıda Tekrarlar

- **{n}:** Önceki desenin tam olarak n kez tekrarlanması gerektiğini belirtir.
 - Örnek: z{2} ifadesi tam olarak zz dizisini yakalar.
 - Örnek: {12} ifadesi, önceki deseni tam olarak 12 kez eşleştirir.

- **{n, m}**: Belirli bir aralık tanımlar. Önceki desenin en az n , en fazla m kez tekrarlanmasını bekler.
 - **Örnek:** {1, 5} ifadesi, desenin 1 ile 5 arasında (1 ve 5 dahil) tekrarlanması durumunda eşleşir.
- **{n,}**: Alt sınır belirler. Önceki desenin en az n kez veya daha fazla tekrarlanması gerektiğini belirtir.
 - **Örnek:** {2,} ifadesi, 2 veya daha fazla eşleşmeyi yakalar.

Esnek Tekrarlar (Kısayollar)

Belirli sayı belirtmek yerine kullanılan genel geçer tekrar sembolleridir:

- ***** (**Asterisk**): Önceki desenin **0 veya daha fazla** kez geçmesi durumunda eşleşir. Yani karakter hiç olmayabilir ya da sınırsız sayıda olabilir.
- **+** (**Plus**): Önceki desenin **1 veya daha fazla** kez geçmesi durumunda eşleşir. Desenin en az bir kez mevcut olması zorunludur.

Düzenli İfadelerde (Regex) Konum Belirleyiciler ve Gruplandırma

Regex kullanımı sadece karakterleri bulmakla sınırlı değildir; bu karakterlerin satırın neresinde (başında mı, sonunda mı) olduğu ve belirli alternatifler arasında nasıl seçim yapılacağı da hayati önem taşır.

3. Konum Belirleyiciler (Anchors)

Bir desenin satırın başında mı yoksa sonunda mı aranacağını belirlemek için kullanılan sembollerdir. Bu semboller karakterin kendisini değil, **pozisyonunu** temsil eder.

- **^** (**Şapka/Hat**) - **Satır Başlangıcı**: Desenin satırın en başında olması gerektiğini belirtir.
 - **Örnek:** ^abc deseni, sadece "abc" ile başlayan satırları yakalar. Satırın ortasındaki bir "abc" ile eşleşmez.
 - **Kritik Not:** ^ sembolünün iki farklı işlevi vardır ve karıştırılmamalıdır:
 1. Eğer [^...] şeklinde **köşeli parantez içinde** kullanılırsa, içerideki karakterleri **hariç tutmak** (exclude) için kullanılır.
 2. Eğer köşeli parantez dışında, desenin en başında kullanılırsa **satır başlangıcını** ifade eder.
- **\$** (**Dolar**) - **Satır Sonu**: Desenin satırın en sonunda olması gerektiğini belirtir.
 - **Örnek:** xyz\$ deseni, sadece "xyz" ile biten satırları yakalar.

4. Gruplandırma ve Alternatifler (Groups & Alternatives)

Karmaşık desenleri bir araya getirmek veya "ya da" mantığını kullanmak için **parantezler** ve **pipe (|)** işareti kullanılır.

Gruplandırma ()

Deseni parantez içine almak, o kısmı tek bir birim olarak ele almanızı sağlar. Bu yapı özellikle tekrarlarda çok kullanışlıdır.

- Tekrar Örneği:** `(no){5}` deseni, "no" hecesini bir bütün olarak ele alır ve yan yana 5 kez tekrar etmesini bekler. Sonuç olarak `nonononono` dizisiyle eşleşir. (Eğer parantez olmasaydı, `no{5}` sadece 'o' harfinin 5 kez tekrarını arardı).

Alternatifler / "Veya" Mantiğı |

Birden fazla seçenekten herhangi birinin eşleşmesini sağlamak için kullanılır.

- Kullanım:** `(secenek1|secenek2)` yapısı, "secenek1" veya "secenek2" ifadelerinden herhangi birini yakalar.
- Senaryo Örneği:** Bir metinde zaman dilimlerini yakalamak istediğinizi düşünelim.
 - Desen:** `during the (day|night)`
 - Bu desen, hem `during the day` hem de `during the night` cümlelerini başarıyla yakalayacaktır.

Özet Tablo: Konum ve Grup Sembolleri

Sembol	Tanım	Örnek	Açıklama
<code>^</code>	Başlangıç	<code>^Merhaba</code>	Satır "Merhaba" ile başlamalı.
<code>\$</code>	Bitiş	<code>Hoşçakal\$</code>	Satır "Hoşçakal" ile bitmeli.
<code>()</code>	Gruplandırma	<code>(abc)+</code>	"abc" grubunun bir veya daha fazla tekrarı.
<code> </code>	Veya (Or)	<code>^(A B)</code>	